

Technisches Design Dokument

Projekt: Deal Or No Deal

Dozentin: Diana Hristova

Gruppe: Fazeleh Daneshmandi, Felix Kuhlemann,
Aleksy Leinweber



Inhaltsverzeichnis

1.Beschreibung.....	2
1.1 Grundfunktionen.....	2
1.2 Technische Funktionen.....	3
1.3 UML (Unified Modeling Language).....	3
1.4 Teilaufgaben	4
2.Probleme.....	5
2.1 Runden Ablaufs Logik.....	5
2.2 Runden Definition und Koffer Ziehungen	7
2.3 Zuweisung von Werten der jeweiligen Koffer.....	8
2.3 Nice To Have (Nicht implementiert)	11
3.Zeitmanagement.....	13
3.1 Trello.....	14
3.2 Meetings.....	14
4.Refactoring	15

1. Beschreibung

1.1 Grundfunktionen

- Begrüßungsbildschirm mit kurzer Erklärung des Spiels.
- Eingabe des Spielernamens.
- Erstellung von 26 Koffern zu Beginn jedes Spiels.
- Zufällige Verteilung der Geldbeträge auf die Koffer.
- Anzeige aller verfügbaren Koffer.
- Auswahl eines persönlichen Koffers durch den Spieler.
- Öffnen von Koffern gemäß den Spielregeln:
- Runde 1: 6 Koffer
 1. Runde 2: 5 Koffer
 2. Runde 3: 4 Koffer
 3. Runde 4: 3 Koffer
 4. Runde 5: 2 Koffer
- Danach jeweils 1 Koffer pro Runde.
- Aktualisierung der Liste verfügbarer Koffer nach jeder Auswahl.
- Anzeige der bereits geöffneten Koffer und ihrer Beträge.
- Berechnung eines Bankangebots nach jeder Runde.
- Entscheidung des Spielers: „Deal“ oder „No Deal“.
- Spielende bei Annahme des Angebots oder nach Öffnen aller Koffer.
- Anzeige des gewonnenen Betrags.

1.2 Technische Funktionen

Verwendung von Klassen zur Strukturierung des Programms.

Nutzung von ArrayList zur Verwaltung der Koffer.

Zufällige Wertzuweisung mittels Collections.shuffle().

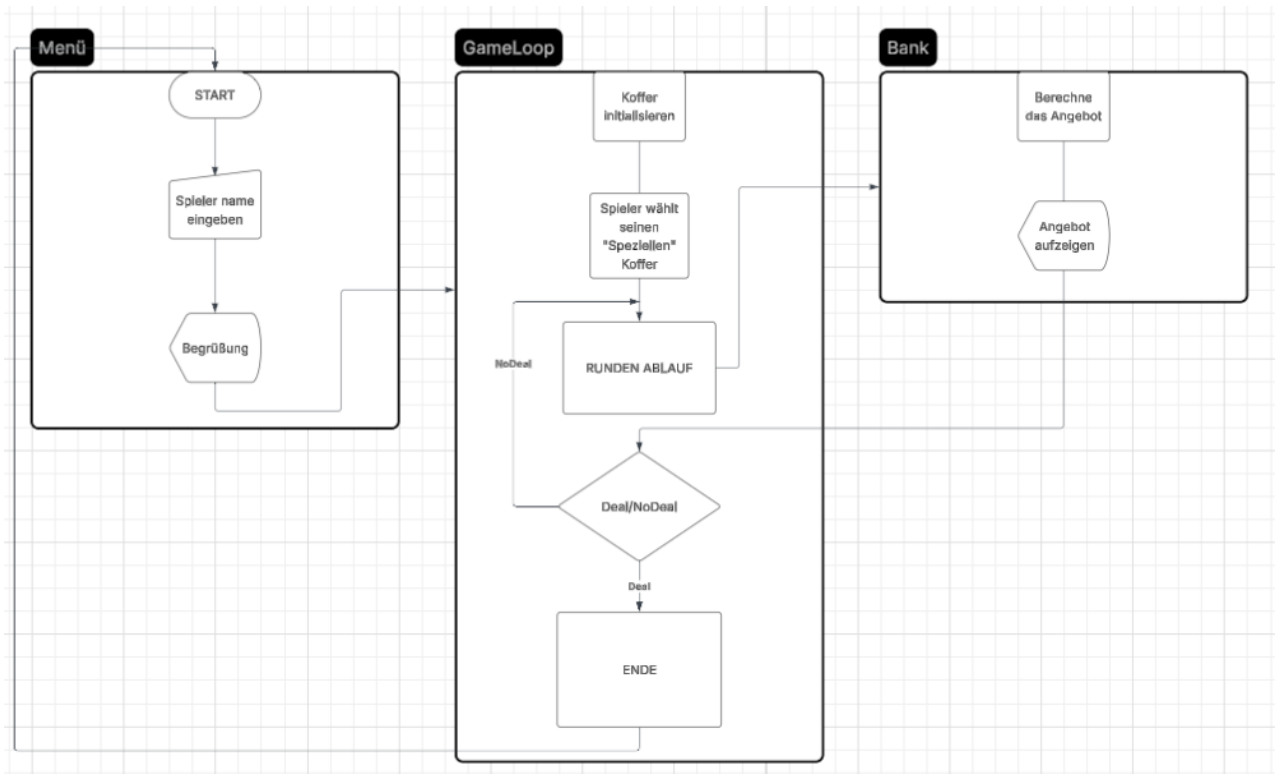
Benutzereingaben mit Scanner.

Spielschleife (Game Loop) zur Steuerung des Spielablaufs.

Überprüfung ungültiger Eingaben (z. B. bereits geöffnete Koffer).

1.3 UML (Unified Modeling Language)

Hier sieht man ein Ablaufs Diagramm, welches eine kleine Skizze darstellen soll, wie das Programm aufgebaut ist.



Eigene Darstellung von der Ablaufs Logik des Programmes¹

¹ [Blank diagram: Lucidchart](#)

1.4 Teilaufgaben

Menü erstellen	Klasse Menue mit main()-Methode und Begrüßungstext
Spielernamen einlesen	Scanner für String-Eingabe
Koffer erstellen Klasse	Koffer mit Nummer, Geldbetrag und Status (geöffnet/nicht geöffnet)
Geldbeträge speichern	ArrayList<Double> mit allen Gewinnwerten
Zufällige Verteilung	Collections.shuffle() auf der Betragsliste
Koffern Werte zuweisen	Schleife, die jedem Koffer einen Betrag zuordnet
Kofferauswahl des Spielers	Scanner für die Eingabe der Koffernummer
Verfügbare Koffer anzeigen	Schleife über alle Koffer, nur ungeöffnete anzeigen
Koffer öffnen	Methode oeffneKoffer()
Rundenlogik umsetzen	Variable aktuelleRunde und switch bzw. Array für Anzahl der zu öffnende Koffer.
Geöffnete Koffer verwalten	Boolean-Attribut geoeffnet in der Klasse Koffer
Bankangebot berechnen	Methode berechneAngebot() mit Formel: $(\text{Summe verbleibender Beträge} / \text{Anzahl verbleibender Koffer}) * (0.1 + (\text{aktuelleRunde} - 1) * 0.1)$
Deal/No Deal-Abfrage	Scanner für Ja/Nein-Eingabe
Spielende prüfen	Wenn Deal angenommen oder letzter Koffer geöffnet wurde
Ergebnis anzeigen	Ausgabe des Gewinns und Vergleich mit eigenem Koffer

Selbsterstellte Tabelle

2. Probleme

2.1 Runden Ablaufs Logik

Das Öffnen von Koffern über den Index Abfrage ist deutlich weniger Intuitiver für den Spieler, als wenn man selbst den Koffern Namen eintippen würde.

```
System.out.println("Runde " + runden);
for (int i = 0; i < anzahlKofferZiehungen; i++) {

    System.out.println("Ungeöffnete Koffer: " + ungeoffneteKoffer);

    System.out.print("Wählen sie bitte ihre Koffernummer aus: ");
    String playerInput = scanner.nextLine();

    int kofferNummer = Integer.parseInt(playerInput);
    int index = kofferNummer - 1;

    if (index >= 0 && index < ungeoffneteKoffer.size()) {
        String koffer = ungeoffneteKoffer.remove(index);
        geoffneteKoffer.add(koffer);
    } else {
        System.out.println("Ungültige Eingabe");
        i--; // Wiederholung derselben Ziehung
    }

    System.out.println("Geöffnete Koffer: " + geoffneteKoffer);
}
```

Über die String Eingabe würde der Code deutlich übersichtlicher aussehen und viel Intuitiver wirken für den Spieler, da man sich den Index für den jeweiligen Koffer nicht merken muss.

```
System.out.println("Runde " + runden);
for (int i = 0; i < anzahlKofferZiehungen; i++) {

    System.out.println("Ungeöffnete Koffer: " + ungeoffneteKoffer);

    System.out.print("Welchen Koffer möchten Sie öffnen? ");
    String eingabe = scanner.nextLine();

    if (ungeoffneteKoffer.contains(eingabe)) {
        ungeoffneteKoffer.remove(eingabe);
        geoffneteKoffer.add(eingabe);
    } else {
        System.out.println("Dieser Koffer existiert nicht oder wurde bereits geöffnet.");
    }

    System.out.println("Geöffnete Koffer: " + geoffneteKoffer);
}
```

Nun besteht das Problem, das man in die zweiten Runden auch mit falschen Koffer Eingaben gelangen kann, da die for schleife nicht aufhört zu zählen. Um das Problem zu beheben, müsste man den Zähler bei falschen eingaben verringern mit "i--".

```
for (int i = 0; i < anzahlKofferZiehungen; i++) {  
    System.out.println("Ungeöffnete Koffer: " + ungeoffneteKoffer);  
    System.out.print("Welchen Koffer möchten Sie öffnen? ");  
    String eingabe = scanner.nextLine();  
    if (ungeoffneteKoffer.contains(eingabe)) {  
        ungeoffneteKoffer.remove(eingabe);  
        geoffneteKoffer.add(eingabe);  
    } else {  
        System.out.println("Dieser Koffer existiert nicht oder wurde bereits geöffnet.");  
        i--;  
    }  
}
```

2.2 Runden Definition und Koffer Ziehungen

Die eher vielleicht dem Anschein nach einfacherer Variante, aber dennoch ein wenig tieferes Verständnis benötigt. Sprich wir hätten bis zu fünften Runden angefangen von 6 Koffern immer weniger Ziehungen. Der Wert 7 ist auch Hardcoded und hat keine wirkliche Bedeutung hier, außer schnell eine Lösung zu generieren. Von 7 würden wir immer die jeweilige Runde abziehen, um somit die Anzahl der Koffer Ziehungen zu erhalten. Ab der fünften Runde würden wir immer nur ein Koffer ziehen. Diese Variante ist hardcoded und weniger verständlich auf den ersten Blick.

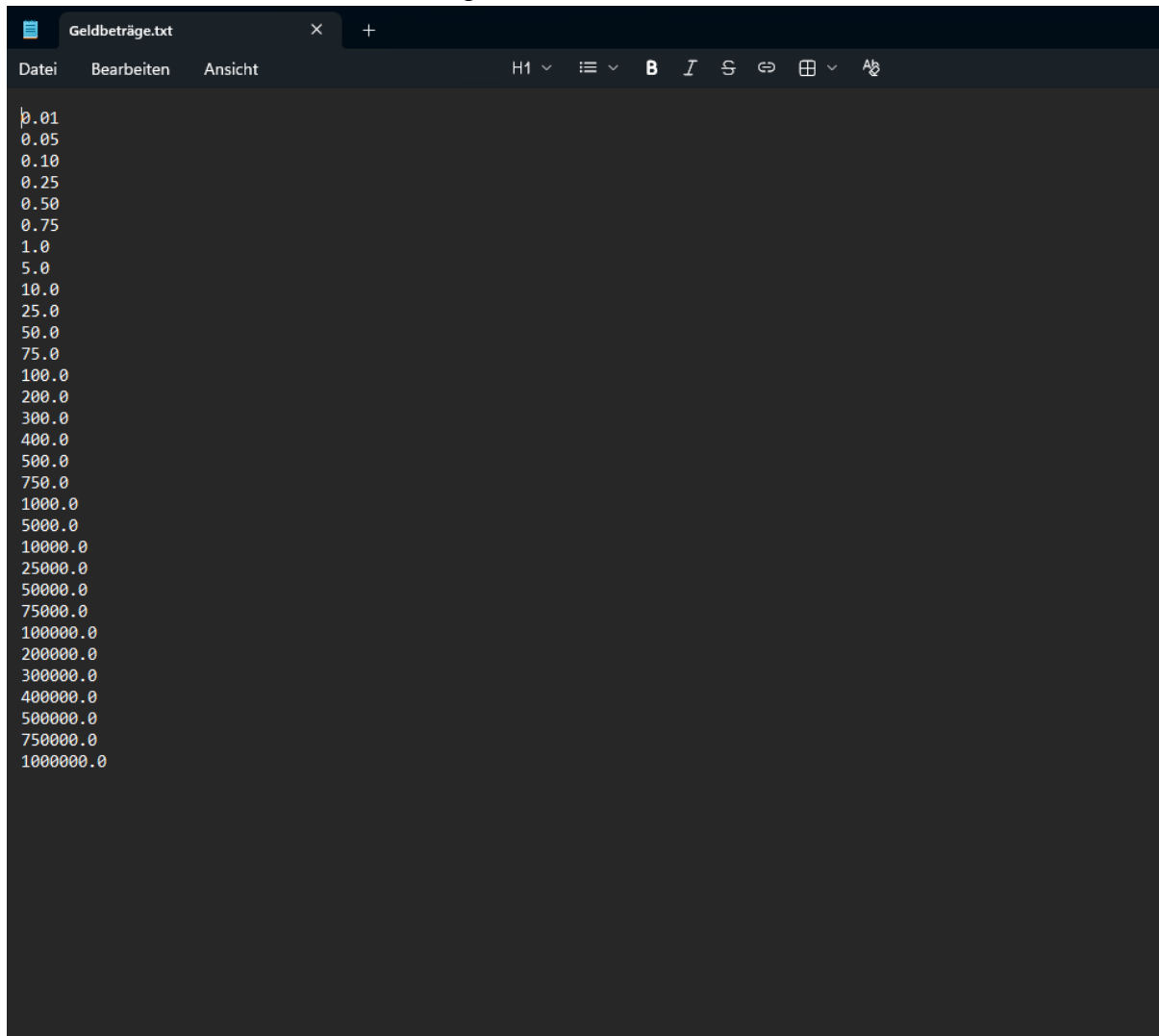
```
//Definition von runden und der jeweiligen Kofferziehung  
int anzahlKofferZiehungen = 0;  
if (runden <= 5) {  
    anzahlKofferZiehungen = 7 - runden;  
} else {  
    anzahlKofferZiehungen = 1;  
}
```

Das Switch-Statement wiederum ist viel logischer von der Struktur her. Hier sehen wir ganz genau das in der ersten Runde wir 6 Koffer ziehen müssen und in der zweiten Runde 5 usw. Ab case (Runde) 5 decken wir alle weiteren Runden mit dem "default" case ab. Also wird jeweils ab runde 5 immer nur ein Koffer gezogen.

```
//Definition von runden und der jeweiligen Kofferziehung
int anzahlKofferZiehungen = 0;
switch (runden) {
case 1:
    anzahlKofferZiehungen = 6;
    break;
case 2:
    anzahlKofferZiehungen = 5;
    break;
case 3:
    anzahlKofferZiehungen = 4;
    break;
case 4:
    anzahlKofferZiehungen = 3;
    break;
case 5:
    anzahlKofferZiehungen = 2;
    break;
default:
    anzahlKofferZiehungen = 1;
    break;
}
```

2.3 Zuweisung von Werten der jeweiligen Koffer

Da das Spiel keine Zufälligen Werte bekommt für die jeweiligen Koffern, sollten daher diese extra vordefiniert Werten. Die Werte sollten dann zufällig verteilt werden an die jeweiligen 26 Koffer sowie den Privaten Koffer, den man am Anfang ja ausgewählt hat. Da aber es sehr unübersichtlich im code sein kann, wenn man `betrage.add(0.01)` usw. bis zur 1.000.000 schreibt, kam der Gedanke auf eine Textdatei zu haben die dort die jeweiligen Werte hat, die jederzeit verändert werden können, ohne auf den Code zuzugreifen.



```
Geldbeträge.txt
Datei Bearbeiten Ansicht H1  B I S ↺ ↻  Aa
0.01
0.05
0.10
0.25
0.50
0.75
1.0
5.0
10.0
25.0
50.0
75.0
100.0
200.0
300.0
400.0
500.0
750.0
1000.0
5000.0
10000.0
25000.0
50000.0
75000.0
100000.0
200000.0
300000.0
400000.0
500000.0
750000.0
1000000.0
```

So würde die Methode aussehen, wenn wir die Werte aus der Textdatei auslesen und die Zeilen die als String in der Textdatei sind umwandeln zu Double.

```
private static void InitialisiereGeldbeträge(){
    try (Scanner scan = new Scanner(new File("Geldbeträge.txt"))) {
        while (scan.hasNextLine()) {
            String zeile = scan.nextLine();
            betraege.add(Double.parseDouble(zeile));
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Mit Collections.shuffle(betraege) würden wir die Elemente in der ArrayList betraege, wo wir ja die Werte aus der Textdatei Geldbeträge.txt eingelesen haben und abgespeichert haben nun zufällig verteilen.

```
private static void InitialisiereGeldbeträge(){
    try (Scanner scan = new Scanner(new File("Geldbeträge.txt"))) {
        while (scan.hasNextLine()) {
            String zeile = scan.nextLine();
            betraege.add(Double.parseDouble(zeile));
            Collections.shuffle(betraege);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

In der Methode KofferZiehungen() sollte auch mit indexOf gearbeitet werden, wenn man eine korrekte Wertezuweisung für die jeweiligen Koffer haben möchte. Was indexOf() macht ist es liest den String ein und gibt den jeweiligen Index davon in der ArrayList z.B zurück. Wenn wir also jetzt 26 Koffer haben, die in der ArrayList gespeichert sind, hat jeder Koffer in der Liste ein Index, wenn wir also Koffer1 abfragen würden, hätte es den Index 0, ebenso.

```
if (ungeoffneteKoffer.contains(eingabe)) {
    int index = ungeoffneteKoffer.indexOf(eingabe);

    double wert = betraege.get(index);
    System.out.println("Im " + eingabe + " sind " + wert + " Euro.");

    ungeoffneteKoffer.remove(index);
    betraege.remove(index);
    geoffneteKoffer.add(eingabe);
} else {
    System.err.println("Dieser Koffer existiert nicht oder wurde bereits geöffnet.");
    i--;
}
```

2.3 Nice To Have (Nicht implementiert)

Um den Privaten Koffer von anderen zu unterscheiden und generell, um geöffnete Koffer von Ungeöffneten Koffern in der Konsole zu unterscheiden ist meist schwierig und man kann sich als User verlaufen ohne ein Interface oder klare farbliche Trennungen, daher ist es als sinnvoll betrachtet worden ANSI Escape Codes einzubinden.

```
private static void PrivaterKofferAuswahl() {
    System.out.println("Ungeöffnete Koffer: " + ungeoffneteKoffer);

    System.out.print("Wähle deinen Speziellen Koffer aus: ");
    String spielerKoffer = scan.nextLine();

    if (ungeoffneteKoffer.contains(spielerKoffer)) {
        ungeoffneteKoffer.remove(spielerKoffer);
        System.out.print("\u001B[32m" + "Dein Spezieller Koffer:" + spielerKoffer + "\u001B[0m ");
    }
}
```

Der jeweilige Code von 30-39 steht für die Buchstabenfarbe und alles ab 40-49 für den Hintergrund des Buchstabens.

Color codes

Most terminals support 8 and 16 colors, as well as 256 (8-bit) colors. These colors are set by the user, but have commonly defined meanings.

8-16 Colors

Color Name	Foreground Color Code	Background Color Code
Black	30	40
Red	31	41
Green	32	42
Yellow	33	43
Blue	34	44
Magenta	35	45
Cyan	36	46
White	37	47
Default	39	49

<https://gist.github.com/fnky/458719343aabd01cfb17a3a4f7296797>

"\u001B[32m"

Erklärung:

\u001B = Escape Zeichen (Sagt das gleich danach eine Steueranweisung folgt)

[= Steueranweisung

31 = Parameter (Farbcode)

M = Aktion

\u001B[32m = Setze Textfarbe auf Grün!

Was Terminal sieht: ESC[32mDeinSpeziellerKoffer

Interpretation:

Textfarbe = Grün

Schreibe "DeinSpeziellerKoffer "

Gelb = Spielerkoffer

Rot = Geöffnete Koffer

Cyan = Verfügbare Koffer

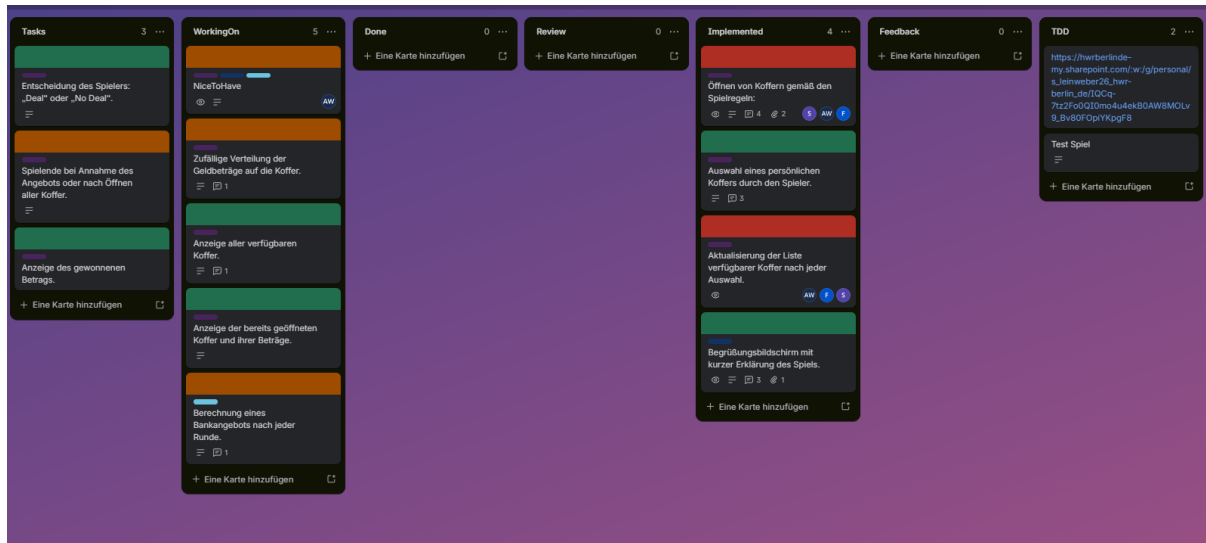
Blau = Rundeninformationen

Grün = Banker-Angebote

3. Zeitmanagement

3.1 Trello

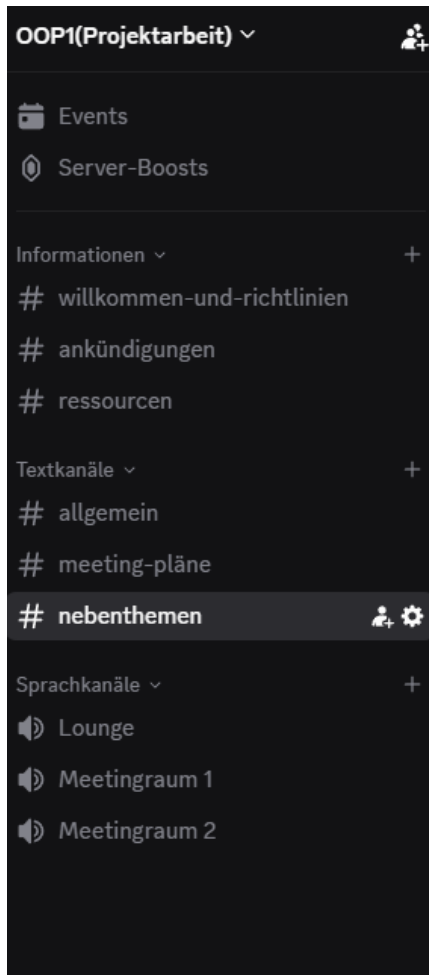
Um zeitlich auch rechtzeitig das Projekt abzugeben haben wir uns auf das Tool Trello geeinigt. Es kann einzelne Aufgaben auflisten mit visueller Unterstützung, sodass man erkennen kann von welcher Schwierigkeit diese Aufgabe ist, um später auch die Priorisierung anzupassen.



Eigene Abbildung vom Trello für das Projekt "Deal or No Deal".

3.2 Meetings

Außerdem hatten wir öfter mal über Discord Online-Meetings gehabt um die Zeit sowie die Umsetzbarkeit auszudiskutieren, um eventuell entstandene Missverständnisse aus dem Weg zu räumen. Dafür haben wir uns einen Discord Server erstellt mit dem jeweiligen Meeting Channel.



Projekt Discord Server

4.Refactoring

Als wir die Geldwertzuweisung für die Koffer als ein Feature angegangen haben, haben wir gemerkt das wir doch über Index der einzelnen Koffer arbeiten müssen. Denn sonst hätten wir Probleme mit den Öffnungen der Koffer, wenn wir über String versucht hätten die Koffer zu öffnen, also nur über `String eingabe = scanner.nextLine()` zum Beispiel. Stattdessen haben wir jetzt `eingabe = Integer.parseInt(scanner.nextLine());` Was uns ermöglicht die Koffer direkt über eine Ganzzahl(Index) zu öffnen.

Zugriffsmöglichkeiten von Variablen wurden angepasst je nach dem Nutzen.

Zugriffsmöglichkeiten auf die Methoden wurden ebenso angepasst.